

A. Ranking (8 points)

The following ranked lists report salaries of employees (in K€) and their departmental budgets (in M€), respectively, in descending order.

<i>salary</i> (K€)	<i>budget</i> (M€)
Dan: 60.0	Pam: 40.0
Abe: 50.0	Joe: 30.0
Pam: 30.0	Dan: 30.0
Joe: 20.0	Abe: 10.0

1. Use TA to find the top-1 employee according to the salary/budget ratio.
 2. Use now FA for the same purpose.
 3. Discuss correctness of TA and FA for this problem: *a)* using the data shown above; *b)* in general.
- In Steps 1 and 2, provide enough details for us to check correctness of your execution with TA and FA.

Solution.

1. With TA, we make one sorted access to *salary* retrieving Dan (60), and one random access to *budget* to retrieve Dan's budget (30); symmetrically, one s.a. to *budget* (Pam, 40) and a corresponding r.a. to *salary* (30). With this, Dan's score is 2, Pam's is 0.75, while the threshold point (60,40) has a score of 1.5. So TA stops and recognizes Dan as the top-1 employee.
2. With FA, we make sorted accesses to both lists until a common object is found, which happens at depth 3 (both Dan and Pam are in common, so their score can be computed); when the sorted access phase stops, all employees have been seen. For Abe and Joe, FA determines their missing partial score through random access. The highest score is Abe's (5), which is the top-1 employee by FA.
3. Clearly, TA fails to recognize the top employee (Abe), while FA succeeds. The problem is the use of a scoring function that is not monotonic in its arguments, so the determination of the threshold is meaningless, particularly because budgets decrease, so overall scores may increase. However, FA may also fail with this scoring function: if we swapped Pam's and Dan's salaries, then Pam would be found on both lists at depth 1 and FA would stop immediately, failing to recognize that Abe has a much higher score.

B. Physical databases (8 points + bonus)

Table *Employee*(ID, Name, HireDate, Salary, ManagerID, Site) stores 30K tuples in an entry-sequenced primary sequential organization with 2K blocks. Evaluate the query plan and cost for the query below in each of the following two scenarios, assuming that val(Site)=60 and that 5% of the employees are top managers (i.e., their ManagerID is NULL); there can only be either top managers or regular employees (non-managers).

1. No auxiliary structures;
2. Secondary indexes: B+(ManagerID) with 500 leaves, B+(Site) with 600 leaves, both with 3 levels, F=70.

```
SELECT * FROM Employee
WHERE ManagerID IN (SELECT ID FROM Employee WHERE Site='Milan' AND ManagerID IS NULL)
```

Bonus (1 extra point): how would the results change if we replaced the first line of the query with **UPDATE Employee SET Salary = 1.1 * Salary**? Motivate your answer.

Solution.

Scenario 1 Start from the nested query: scan the 2K blocks of the primary structure. During the scan find the tuples where Site='Milan' and ManagerID IS NULL: they are $30K * 5\% * 1/60 = 25$ tuples. Cache these 25 tuples (managers from Milan), in particular their IDs.

For the external query: scan again the 2K blocks of the primary structure and find the employees where ManagerID in the set of the 25 managers IDs.

TOTAL COST = 2K + 2K = 4K I/O operations

Scenario 2 Start from B+(Site) and search for 'Milan': this requires to read 2 intermediate nodes + all the leaf nodes that contain the pointers to 'Milan' employees. There are $30K/60=500$ employees from Milan. A leaf node can contain $30K/600=50$ pointers. To read 500 pointers we need to read 10 leaf nodes (rounded to 11). Then, we follow the 500 pointers and check if the ManagerID IS NULL. We will find 25 managers ($500*5\%$). We cache them.

For the external query one may proceed like in scenario 1, but we can optimize the plan by exploiting also the B+(ManagerID). For each top manager from Milan we can search the ID in the B+(ManagerID). This has 2 intermediate levels. For each ManagerID we expect to have $95\% / 5\% = 19$ employees. Each leaf node can contain $30K/500$ pointers, therefore the 19 pointers in most of the cases will be in a single leaf

node; we need to read 3 nodes in the B+ tree. For each `ManagerID`, we follow the 19 pointers to find the full tuples of the employees that have that ID in the `ManagerID` field to be returned in the select clause.

TOTAL COST = 2 (int. levels) + 11 (leaf nodes) + 500 pointers (to reach the full tuples of the 'Milan' employees) + 25 * (3 (levels in B+(`ManagerID`)) + 19 (pointers)) = 1063 I/O operations

If the plan starts from B+(`ManagerID`) the cost for the nested query becomes: 2 intermediate nodes + (1500/60 +1 = 26) = 28 leaf nodes; then, follow 30K*5%=1500 pointers to reach the full tuples to check if the site is 'Milan'. This cost (2+28+1500) is higher than (2+11+500).

Bonus: Updating a tuple requires the containing block to be read and then *overwritten*; we can assume that both the costs of reading and overwriting can be measured as 1 I/O, for simplicity. If we replace the first part with an update we have to do exactly the same steps described before, we just need to add the "write part": for all the blocks that contain an employee with a Manager from 'Milan' (they are 25*19 = 475), the salary is updated and the block must be persisted into secondary memory. In the worst case, the 475 tuples are contained in 475 different blocks: this is an upper-bound for the cost that must be added to the costs of the two previous scenarios that already consider the "read part" of the update.

C. Trigger (8 points)

A DBMS manages the enrolment of students in the current exam call, using the following two tables:

ENROLLED_STUDENT(`StudentID`, `Name`, `AssignedRoom`) VIRTUAL_ROOM(`RoomID`, `NumberOfStudents`)

Table VIRTUAL_ROOM keeps a list of virtual rooms with the number of students assigned to them. A virtual room can hold up to 25 students. When a student unsubscribes, the corresponding row is deleted from ENROLLED_STUDENT; the system then checks whether it can merge two classes into one with at most 25 students. Virtual rooms with 0 students are deleted. Write a set of triggers enforcing the above policy when reacting to unsubscribe events.

Solution.

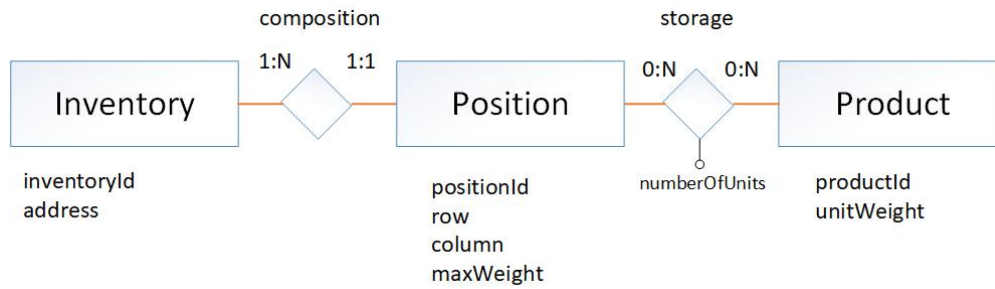
```
CREATE TRIGGER T1
AFTER DELETE ON ENROLLED_STUDENT
FOR EACH ROW
BEGIN
UPDATE VIRTUAL_ROOM
SET NumberOfStudents = NumberOfStudents - 1
WHERE RoomID = old.assignedRoom;
END

CREATE TRIGGER T2
AFTER UPDATE OF NumberOfStudents ON VIRTUAL_ROOM
FOR EACH ROW
WHEN new.NumberOfStudents = 0
BEGIN
DELETE FROM virtual_room WHERE RoomID=new.RoomID;
END

CREATE TRIGGER T3
AFTER UPDATE OF NumberOfStudents ON VIRTUAL_ROOM
FOR EACH ROW
WHEN new.NumberOfStudents > 0 AND new.NumberOfStudents < old.NumberOfStudents
BEGIN
DECLARE R INT;
SELECT IFNULL((SELECT RoomID FROM VIRTUAL_ROOM WHERE NumberOfStudents <= (25 - new.NumberOfStudents)
AND RoomID<>new.RoomID LIMIT 1),0) INTO R;
IF (R<>0) THEN
UPDATE ENROLLED_STUDENTS SET AssignedRoom=R WHERE AssignedRoom=new.RoomID;
UPDATE VIRTUAL_ROOM SET NumberOfStudents = NumberOfStudents + new.NumberOfStudents WHERE RoomID = R;
DELETE FROM VIRTUAL_ROOM WHERE RoomID = new.RoomID;
END IF;
END
```

D. JPA (8 points)

An application manages the storage of goods in a repository. Data comply with the conceptual model below.



The user can inspect any one of the inventories and browse the positions. By selecting a position, the user can check the current status of occupancy (empty or occupied by one or more products, total weight of the occupants, etc). The user can also search for a product Id and see the positions where it is stored and the inventory corresponding to each such position.

Products are in the order of hundreds of thousands, inventories are in the order of ten and positions per inventory in the order of thousands.

Show the JPA entities (with all their attributes) that map the domain objects of the conceptual model, taking into account the above mentioned access paths of the application and data cardinalities. When designing the annotations for the relationships, specify the owner side of the relationship, the mapped-by attribute, the fetch policy and the cascading policies you consider more appropriate to support the access required by the web application. Comment on the design choices you have made.

Solution.

See separate file with the solution.